

ROS2

DAY 4 – Manual 2

ROS2 Services – Server & Client Implementation

Objective

By the end of Manual 2, you will be able to:

1. Understand each line of the ROS2 service server and client Python code.
2. Run and test ROS2 services step-by-step.
3. Debug common service-related issues.

Step 0 – Pre-Requisites

- Complete Manual 1 setup: `service_pkg` and `my_py_pkg` created, built, and sourced.
- ROS2 workspace sourced:

```
source ~/ros2_ws/install/setup.bash
```

- Basic understanding of Python, ROS2 nodes, and topics (Day 1 & 2).

Step 1 – Service Server Node (`srv_server.py`)

1.1 Import Required Modules

```
import rclpy
from rclpy.node import Node
from service_pkg.srv import TestService
```

- `rclpy` → ROS2 Python client library.
- `Node` → Base class to create ROS2 nodes.
- `TestService` → Service interface imported from `service_pkg` (generated from `.srv`).

1.2 Define Server Node Class

```
class ServiceServer(Node):  
    def __init__(self):  
        super().__init__('ServiceServerNode')
```

- ServiceServer inherits from Node.
- 'ServiceServerNode' → name of the node shown in ros2 node list.

1.3 Create the Service

```
self.srv = self.create_service(TestService, 'test_service', self.callback)
```

- create_service() registers a service:
 - TestService → type of service
 - 'test_service' → name of the service (used by clients to call)
 - self.callback → function executed on each request

1.4 Log Service Ready

```
self.get_logger().info("Service Server Ready: test_service")
```

- Prints a message to console indicating server is ready.

1.5 Callback Function

```
def callback(self, request, response):  
    self.get_logger().info(f"Incoming request: {request.input}")  
    response.output = "hello client... Got your req"  
    self.get_logger().info(f"Sending response: {response.output}")  
    return response
```

Line-by-Line Explanation:

1. request → object containing client request data (request.input)
 2. Log the request for visibility.
 3. Assign a response string to response.output.
 4. Log the response.
 5. Return response → this is sent back to the client.
-

1.6 Main Function

```
def main(args=None):  
    rclpy.init(args=args)  
    node = ServiceServer()  
    rclpy.spin(node)  
    rclpy.shutdown()
```

- rclpy.init() → initialize ROS2 Python library
 - rclpy.spin(node) → keeps node alive to process incoming requests
 - rclpy.shutdown() → clean shutdown after node exits
-

1.7 Run Guard

```
if __name__ == '__main__':  
    main()
```

- Ensures this script runs only if executed directly, not on import.
-

Step 2 – Service Client Node (srv_client.py)

2.1 Import Required Modules

```
import rclpy  
from rclpy.node import Node  
from service_pkg.srv import TestService
```

- Same as server, needed for client functionality.

2.2 Define Client Node Class

```
class ServiceClient(Node):  
    def __init__(self):  
        super().__init__('ServiceClientNode')
```

- Creates a client node named 'ServiceClientNode'.

2.3 Create Client Object

```
self.client = self.create_client(TestService, 'test_service')  
while not self.client.wait_for_service(timeout_sec=1.0):  
    self.get_logger().warn("Service not available... waiting")
```

- `create_client()` → attaches this node to the service
- Waits until the service is available before sending requests
- Timeout 1 sec → prints warning if service not ready

2.4 Send Request Function

```
def send_request(self, req):  
    request = TestService.Request()  
    request.input = req  
    future = self.client.call_async(request)  
    rclpy.spin_until_future_complete(self, future)  
    return future.result()
```

Line-by-Line:

1. `TestService.Request()` → create a request object
 2. `request.input = req` → assign the string to send
 3. `call_async(request)` → send request asynchronously
 4. `spin_until_future_complete()` → wait for response
 5. `future.result()` → returns the server response object
-

2.5 Main Function

```
def main(args=None):
    rclpy.init(args=args)
    client = ServiceClient()
    response = client.send_request("Hello server")
    print(f"Result from server: {response.output}")
    rclpy.shutdown()
```

- Initializes ROS2, creates client node
 - Sends "Hello server" request to test_service
 - Prints server's response
 - Shutdown cleanly
-

2.6 Run Guard

```
if __name__ == '__main__':
    main()
```

- Ensures proper execution when script is run directly.
-

Step 3 – Build the Workspace

1. Build interface package first:

```
cd ~/ros2_ws
colcon build --packages-select service_pkg
source install/setup.bash
```

2. Build Python nodes:

```
colcon build --packages-select my_py_pkg
source install/setup.bash
```

Step 4 – Run and Test the Service

1. Open Terminal 1 – run server:

```
ros2 run my_py_pkg srv_server
```

- Output:

```
[INFO] [ServiceServerNode]: Service Server Ready: test_service
```

2. Open Terminal 2 – run client:

```
ros2 run my_py_pkg srv_client
```

- Output:

```
[INFO] [ServiceClientNode]: Service not available... waiting # only if server not ready  
Result from server: hello client... Got your req
```

3. Terminal 1 logs:

```
Incoming request: Hello server  
Sending response: hello client... Got your req
```

Step 5 – Common Issues

- Service not available → likely server not running or workspace not sourced.
 - ImportError: TestService → check that service_pkg was built first and sourced.
 - Timeouts → client waits until server is ready; always run server first.
-

Step 6 – Concepts Recap

- service_pkg → defines .srv interface, CMake-based
 - my_py_pkg → Python nodes using service
 - Server waits for requests, client sends requests, synchronous communication
 - Each Python node line explained
 - Always build interface first, then Python nodes
-